OS Lab

Skill-17

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50
#define MAX_JOBS 20

typedef struct
{
   int job_id;
   pid_t pid;
   pid_t pgid;
   char command[MAX_INPUT];
   char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

pid_t shell_pgid;
int shell_terminal;


// ======================================
// FIND JOB
// ======================================
```

```
int find_job(int job_id)
{
   for (int i = 0; i < job_count; i++)
   {
      if (jobs[i].job_id == job_id)
      {
         return i;
      }
   }

   return -1;
}


// ======================================
// ADD JOB
// ======================================

void add_job(pid_t pid, pid_t pgid, char *command)
{
   if (job_count >= MAX_JOBS)
   {
      printf("Job table is full\n");
      return;
   }

   jobs[job_count].job_id = next_job_id++;
   jobs[job_count].pid = pid;
   jobs[job_count].pgid = pgid;

   strcpy(jobs[job_count].command, command);
   strcpy(jobs[job_count].state, "Running");

   job_count++;
}


// ======================================
// REMOVE JOB
// ======================================

void remove_job(int index)
{
   for (int i = index; i < job_count - 1; i++)
```

```
    {
        jobs[i] = jobs[i + 1];
    }

    job_count--;
}


// ======================================
// UPDATE JOBS
// ======================================

void update_jobs()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED
        );

        if (result == jobs[i].pid)
        {
            if (WIFEXITED(status) ||
                WIFSIGNALED(status))
            {
                printf(
                    "\n[%d] Done    %s\n",
                    jobs[i].job_id,
                    jobs[i].command
                );

                remove_job(i);
                i--;
            }
            else if (WIFSTOPPED(status))
            {
                strcpy(
                    jobs[i].state,
                    "Stopped"
                );
```

```
        }
      }
    }
}


// ======================================
// LIST JOBS
// ======================================

void list_jobs()
{
    update_jobs();

    if (job_count == 0)
    {
        printf("No active jobs\n");
        return;
    }

    printf("\n");
    printf("Job ID\tPID\tPGID\tState\t\tCommand\n");
    printf("-------------------------------------------------------\n");

    for (int i = 0; i < job_count; i++)
    {
        printf(
            "[%d]\t%d\t%d\t%s\t\t%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].pgid,
            jobs[i].state,
            jobs[i].command
        );
    }

    printf("\n");
}


// ======================================
// BRING JOB TO FOREGROUND
// ======================================
```

```
void foreground_job(int job_id)
{
    int index = find_job(job_id);

    if (index == -1)
    {
        printf("No such job: %d\n", job_id);
        return;
    }

    Job *job = &jobs[index];

    printf(
        "Bringing job [%d] to foreground: %s\n",
        job->job_id,
        job->command
    );

    // Give terminal to job's process group
    tcsetpgrp(
        shell_terminal,
        job->pgid
    );

    // Resume process if stopped
    if (strcmp(job->state, "Stopped") == 0)
    {
        if (kill(-job->pgid, SIGCONT) == -1)
        {
            perror("SIGCONT");
        }

        strcpy(
            job->state,
            "Running"
        );
    }

    int status;

    // Wait for foreground process
    waitpid(
        job->pid,
        &status,
```

```
        WUNTRACED
    );

    // Take terminal back
    tcsetpgrp(
        shell_terminal,
        shell_pgid
    );

    // Process completed
    if (WIFEXITED(status) ||
        WIFSIGNALED(status))
    {
        remove_job(index);
    }

    // Process stopped
    else if (WIFSTOPPED(status))
    {
        strcpy(
            job->state,
            "Stopped"
        );

        printf(
            "\n[%d] Stopped    %s\n",
            job->job_id,
            job->command
        );
    }
}


// ======================================
// EXECUTE COMMAND
// ======================================

void execute_command(char *input)
{
    char input_copy[MAX_INPUT];

    strcpy(input_copy, input);

    char *args[MAX_ARGS];
```

```
int argc = 0;

char *token =
   strtok(input, " \t\n");

while (
   token != NULL &&
   argc < MAX_ARGS - 1
)
{
   args[argc++] = token;

   token = strtok(NULL, " \t\n");
}

args[argc] = NULL;

if (argc == 0)
{
   return;
}

pid_t pid = fork();

if (pid < 0)
{
   perror("fork");
   return;
}

// ====================================
// CHILD
// ====================================

if (pid == 0)
{
   /*
    * Create a new process group.
    */
   setpgid(
      0,
      0
   );
```

```
    /*
     * Restore default signal handling.
     */
    signal(SIGINT, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);
    signal(SIGTTIN, SIG_DFL);
    signal(SIGTTOU, SIG_DFL);

    execvp(
        args[0],
        args
    );

    perror("execvp");
    exit(EXIT_FAILURE);
}

// ====================================
// PARENT
// ====================================

/*
 * Put child into its own process group.
 */
setpgid(
    pid,
    pid
);

/*
 * Give terminal to child.
 */
tcsetpgrp(
    shell_terminal,
    pid
);

int status;

/*
 * Wait until child finishes
 * or is stopped using Ctrl+Z.
 */
waitpid(
```

```
        pid,
        &status,
        WUNTRACED
    );

    /*
     * Take terminal back.
     */
    tcsetpgrp(
        shell_terminal,
        shell_pgid
    );

    // ====================================
    // CHILD STOPPED
    // ====================================

    if (WIFSTOPPED(status))
    {
        add_job(
            pid,
            pid,
            input_copy
        );

        int index =
            job_count - 1;

        strcpy(
            jobs[index].state,
            "Stopped"
        );

        printf(
            "\n[%d] Stopped    %s\n",
            jobs[index].job_id,
            jobs[index].command
        );
    }
}


// ======================================
// INITIALIZE SHELL
```

```
// ========================================

void initialize_shell()
{
   shell_terminal = STDIN_FILENO;

   /*
    * Put shell in its own process group.
    */
   shell_pgid = getpid();

   setpgid(
      shell_pgid,
      shell_pgid
   );

   /*
    * Give terminal to shell.
    */
   tcsetpgrp(
      shell_terminal,
      shell_pgid
   );

   /*
    * Ignore terminal signals in shell.
    */
   signal(SIGINT, SIG_IGN);
   signal(SIGTSTP, SIG_IGN);
   signal(SIGTTIN, SIG_IGN);
   signal(SIGTTOU, SIG_IGN);
}


// ========================================
// MAIN
// ========================================

int main()
{
   char input[MAX_INPUT];

   initialize_shell();
```

```
while (1)
{
    update_jobs();

    printf("jobcontrol$ ");
    fflush(stdout);

    if (
        fgets(
            input,
            sizeof(input),
            stdin
        ) == NULL
    )
    {
        break;
    }

    input[
        strcspn(input, "\n")
    ] = '\0';

    if (strlen(input) == 0)
    {
        continue;
    }

    // ================================
    // EXIT
    // ================================

    if (
        strcmp(
            input,
            "exit"
        ) == 0
    )
    {
        break;
    }

    // ================================
    // JOBS
    // ================================
```

```
        if (
            strcmp(
                input,
                "jobs"
            ) == 0
        )
        {
            list_jobs();
            continue;
        }

        // ================================
        // FOREGROUND
        // Example: fg 1
        // ================================

        if (
            strncmp(
                input,
                "fg ",
                3
            ) == 0
        )
        {
            int job_id =
                atoi(input + 3);

            foreground_job(job_id);

            continue;
        }

        // Normal command
        execute_command(input);
    }

    return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~$ nano job_control.c
lahari@DESKTOP-AIC33F7:~$ gcc job_control.c -o job_control
lahari@DESKTOP-AIC33F7:~$ ./job_control
jobcontrol$ sleep 30
^Z
[1] Stopped    sleep 30
jobcontrol$ jobs

Job ID  PID     PGID    State           Command
---------------------------------------------------------
[1]     783     783     Stopped         sleep 30

jobcontrol$ fg 1
Bringing job [1] to foreground: sleep 30
jobcontrol$ sleep 20
^Z
[2] Stopped    sleep 20
jobcontrol$ jobs

Job ID  PID     PGID    State           Command
---------------------------------------------------------
[2]     789     789     Stopped         sleep 20

jobcontrol$ fg 2
Bringing job [2] to foreground: sleep 20
jobcontrol$ sleep 10
jobcontrol$ ^Z
jobcontrol$ jobs
No active jobs
jobcontrol$ fg 1
No such job: 1
jobcontrol$ fg2
execvp: No such file or directory
jobcontrol$ fg 2
No such job: 2
jobcontrol$
```